

Documento de Requisitos

Sistema de Funcionários

1. Visão geral

Aplicação de console, originalmente escrita em C (main.c), para controle de funcionários de uma empresa. Por meio de um menu em modo texto, o operador cadastra funcionários (nome, cargo e salário), lista todos os registros e busca um funcionário pelo nome. Os dados ficam apenas em memória (até 100 registros) e são perdidos ao encerrar o programa. Este resumo foi extraído da documentação do sistema legado.

2. Requisitos funcionais

ID	Requisito	Prioridade
RF001	Exibir um menu com as opções 1 (Adicionar), 2 (Listar), 3 (Buscar) e 0 (Sair), executar a opção escolhida e repetir o menu até o usuário sair.	Essencial
RF002	Cadastrar um funcionário informando nome, cargo e salário.	Essencial
RF003	Listar todos os funcionários na ordem de cadastro, com número, nome, cargo e salário.	Essencial
RF004	Buscar um funcionário pelo nome e exibir seu cargo e salário.	Essencial
RF005	Encerrar o sistema pela opção 0, exibindo a mensagem 'Encerrando...'	Essencial
RF006	Impedir novos cadastros ao atingir 100 funcionários, exibindo 'Limite atingido!'.	Importante
RF007	Exibir 'Opção inválida!' quando o número digitado não corresponder a uma opção do menu.	Importante

Melhorias previstas (evolução)

ID	Requisito	Prioridade
RF008	Editar funcionário.	Importante
RF009	Remover funcionário.	Importante

ID	Requisito	Prioridade
RF010	Gerar identificador (ID) único para cada funcionário.	Importante
RF011	Validar entradas (opção numérica, salário maior que 0, nome não vazio).	Importante
RF012	Busca parcial e sem diferenciar maiúsculas de minúsculas.	Desejável
RF013	Relatório com total, soma e média salarial.	Desejável

3. Requisitos não funcionais

ID	Categoria	Requisito	Situação atual
NF001	Usabilidade	Interface em menu numerado, com mensagens claras em português.	Atendido
NF002	Desempenho	Cadastro, listagem e busca imediatos para até 100 registros (busca linear).	Atendido
NF003	Confiabilidade	Leitura de textos sem estouro de buffer (fgets com tamanho limitado).	Atendido
NF004	Confiabilidade	Tolerar entrada não numérica no menu e salários inválidos sem travar.	Não atendido
NF005	Portabilidade	Usar apenas C padrão (stdio.h, string.h), compilável em qualquer sistema.	Atendido
NF006	Capacidade	Armazenar até 100 funcionários em memória (vetor estático).	Atendido
NF007	Manutenibilidade	Código organizado em funções por responsabilidade e estrutura de dados única.	Atendido
NF008	Persistência	Os dados devem sobreviver ao encerramento do programa.	Não atendido
NF009	Segurança	Acesso aos dados salariais controlado por autenticação.	Não atendido

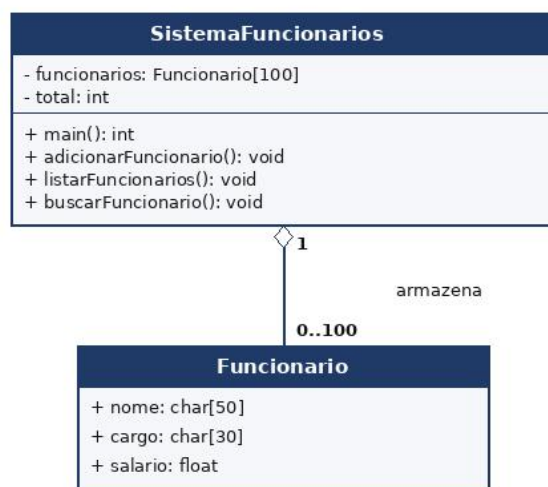
ID	Categoria	Requisito	Situação atual
NF010	Persistência	Persistir os dados em banco de dados (evolução).	Previsto

4. Regras de negócio

ID	Regra
RN01	A capacidade máxima é de 100 funcionários.
RN02	Todo funcionário possui nome, cargo e salário.
RN03	A busca exige nome idêntico (diferencia maiúsculas, minúsculas e acentos) e retorna apenas o primeiro registro encontrado.
RN04	A numeração da listagem é a posição de cadastro, não um identificador.
RN05	O salário é exibido em reais com duas casas decimais.
RN06	Nomes duplicados são aceitos.

Limites do código: nome com até 49 caracteres e cargo com até 29 caracteres.

5. Diagrama de classes



O código original é procedural; a estrutura de dados e o conjunto de funções são representados como classes apenas para documentação. SistemaFuncionarios armazena de 0 a 100 objetos Funcionario.

6. Entidades e tabelas possíveis

Versão mínima (equivalente ao código atual)

Tabela	Campos	Restrições
FUNCIONARIO	id (INTEGER, PK, autoincremento) nome (VARCHAR(50)) cargo (VARCHAR(30)) salario (DECIMAL(10,2))	nome, cargo e salario NOT NULL CHECK (salario >= 0)

Versão ampliada (se o sistema crescer)

Tabela	Campos	Relacionamento
DEPARTAMENTO	id (int, PK) nome (varchar)	1:N com FUNCIONARIO (um departamento possui vários funcionários)
CARGO	id (int, PK) descricao (varchar)	1:N com FUNCIONARIO (um cargo classifica vários funcionários)
FUNCIONARIO	id (int, PK) nome (varchar) salario (decimal) data_admissao (date) ativo (boolean) id_cargo (int, FK) id_departamento (int, FK)	N:1 com CARGO e N:1 com DEPARTAMENTO

Operações do sistema em SQL (versão mínima)

Operação	Comando SQL
Cadastrar	INSERT INTO funcionario ...
Listar	SELECT * FROM funcionario ORDER BY nome
Buscar	SELECT cargo, salario FROM funcionario WHERE nome = ?
Editar (proposto)	UPDATE funcionario SET ... WHERE id = ?
Remover (proposto)	DELETE FROM funcionario WHERE id = ?

7. Bancos de dados possíveis

Banco	Tipo	Indicado para
SQLite	Relacional embarcado	Uso local e trabalho acadêmico; sem servidor; módulo sqlite3 já incluso no Python.
PostgreSQL	Relacional	Uso em empresa, com vários usuários.
MySQL/MariaDB	Relacional	Sistemas web e multiusuário.
SQL Server	Relacional	Ambiente Microsoft.
MongoDB	NoSQL (documentos)	Dados sem esquema fixo (pouco necessário neste caso).
Arquivo JSON/CSV	Arquivo	Primeira etapa de persistência.

Recomendação: SQLite como primeira escolha; PostgreSQL ou MySQL se houver uso multiusuário.